

Pontifícia Universidade Católica de Campinas

Engenharia de Software

Padrões e Arquitetura de Software

ECF-1: Refatoração guiada por SOLID, Clean Code e Padrões GoF

Augusto Fidélis dos Santos Custódio – 24024320

Caio Ribeiro Abrahão – 23006325

João Pedro Pires de Andrade- 23025141

Raphael Fernandes de Sellos Moreira – 1906838

1. Identificação das violações SOLID

Violação de SRP (Princípio da Responsabilidade Única)

O SRP (Single Responsibility Principle) ou Princípio da Responsabilidade Única dita que uma classe deve ter apenas uma única responsabilidade e que mudanças em áreas diferentes do sistema não devem afetar a mesma classe.

A classe `Sis()` do código legado atuava como uma Classe Deus, em contramão completa ao princípio de SRP: uma classe gigante, monolítica com um acúmulo de funções. A mesma classe concentrava:

- **A Infraestrutura e Persistência:** Manipulação direta do driver do SQLite, abertura de cursores e escrita de queries SQL.
- **As Regras de Negócio:** Mapeamento e cálculo de taxas de desconto de itens e tipos de clientes (VIP, Corporativo)
- **A Comunicação/Efeitos Colaterais:** Emissão de saídas de texto simulando notificações de e-mail e SMS para os clientes.

Violação de OCP (Princípio de Aberto/Fechado)

O Princípio Aberto/Fechado diz que o sistema deve estar aberto para extensão, mas fechado para modificação. Em outras palavras, deve ser possível adicionar novos comportamentos criando classes, sem precisar editar código já existente.

No legado, quase toda nova funcionalidade exige editar métodos existentes cheios de `if` e `elif`.

Por exemplo, o cálculo de descontos pelo método `add_ped()` é feito pelo encadeamento `if/elif` gigante, direcionando o tipo de regra de negócio com base no tipo de desconto do item.

Isso torna a implementação de novas funcionalidades arriscada, pois alterações em fluxos antigos podem introduzir bugs em comportamentos que já funcionavam.

```
for i in its:
    if i['tipo'] == 'normal':
        tot += i['p'] * i['q']
    elif i['tipo'] == 'desc10':
        tot += i['p'] * i['q'] * 0.9
    elif i['tipo'] == 'desc20':
        tot += i['p'] * i['q'] * 0.8
    elif i['tipo'] == 'frete_gratis':
        tot += i['p'] * i['q']
```

Violação de LSP (Princípio da Substituição de Liskov)

O LSP afirma que uma subclasse deve poder substituir sua classe base sem quebrar o comportamento esperado do sistema.

A classe `PedEspecial` herda de `Sis`, mas altera comportamentos importantes de forma incompatível. No método `upd_st`, por exemplo, ela permite atualizar o pedido diretamente para qualquer estado, ignorando as transições e notificações existentes na classe pai. O próprio comentário do código indica que o pedido especial “pula direto para qualquer estado”, ignorando transições intermediárias.

Isso é problemático porque um objeto `PedEspecial` não se comporta como um `Sis` comum. Se alguma parte do sistema espera que `upd_st()` envie notificações, calcule pontos ou respeite regras de status, essa expectativa é quebrada ao usar a subclasse.

Violação de ISP (Princípio da Segregação de Interfaces)

Embora o código legado não possua interfaces formais, ele viola a ideia do ISP porque concentra operações muito diferentes em uma única classe. A classe `Sis` obriga qualquer uso do sistema a carregar métodos de pedido, pagamento, estoque, relatório, persistência, cancelamento e notificação no mesmo objeto.

Um módulo que precisasse apenas processar pagamentos acabaria dependendo também de métodos como `gerar_rel`, `validar_estoque`, `calc_tot_cli`, `cancelar_pedido` e acesso direto ao banco. Isso cria uma interface “gorda”, pouco coesa e difícil de reutilizar.

Violação de DIP (Princípio da Inversão de Dependência)

DIP diz que módulos de alto nível não devem depender diretamente de detalhes de baixo nível, mas sim de abstrações.

No código legado, a classe *Sis* depende diretamente do *SQLite*, do sistema de arquivos, de `print()` para notificações e de regras concretas de pagamento. A conexão com o banco é criada diretamente no construtor, usando `sqlite3.connect('loja.db')`, e os métodos executam SQL diretamente dentro da própria classe. Além disso, relatórios são gravados diretamente em arquivos `.txt`, como `rel_vendas.txt` e `rel_clientes.txt`.

Isso viola DIP porque a regra de negócio fica presa a detalhes técnicos. Se fosse necessário trocar *SQLite* por *PostgreSQL*, trocar `print()` por envio real de e-mail, ou trocar arquivo `.txt` por *PDF*, seria preciso alterar a classe principal do sistema.

2. Soluções Implementadas

A solução adotada para consertar a grave violação do Princípio da Responsabilidade Única (SRP) consistiu na Segregação de Responsabilidades por Camadas Coesas (também conhecida como *Separation of Concerns*).

Em vez de manter uma única classe monolítica (*Sis*) que acumulava múltiplos motivos para mudar, o sistema foi totalmente fatiado em componentes especializados:

- **Desacoplamento da Infraestrutura (Camada de Persistência):** classe legada misturava lógicas matemáticas de desconto com comandos SQL, abertura de cursors do *SQLite* e manipulação de transações. A Solução Adotada consiste na criação de *OrderRepository* (implementando a *OrderRepositoryInterface*). Agora, este componente possui o único motivo para mudar: se a estrutura das tabelas ou a tecnologia do banco de dados (*SQLite*) for alterada. Ele não sabe como um desconto é calculado ou quando uma mensagem deve ser enviada; ele apenas traduz objetos de domínio para tabelas relacionais.

- **Criação de Objetos de Domínio Puros (Camada de Modelos):** O estado do pedido e de seus itens trafegava pelo sistema na forma de dicionários e strings primitivas soltas, o que misturava a representação do dado com a execução da regra. Como remediação, as estruturas de dados nas dataclasses *Order* e *OrderItem*, e introduzimos os Enums *OrderStatus*, *CustomerType* e *DiscountType*. Essas classes possuem responsabilidade única sobre o estado das entidades, garantindo que regras de banco ou de envio de mensagens não poluam os modelos de negócio.

- **Centralização do Fluxo de Negócio (Camada de Serviços):** A lógica de criação, pagamento, cancelamento e atualização de pedidos foi deslocada para classes de serviço, como *OrderService*,

PaymentService, DiscountService e NotificationService. Dessa forma, cada serviço passou a representar uma responsabilidade específica do domínio. O OrderService, por exemplo, coordena o fluxo principal do pedido, mas não precisa saber como os dados são persistidos internamente, como cada desconto é calculado em detalhes ou como cada notificação é enviada.

- **Isolamento das Regras de Desconto:** os cálculos de desconto, que antes estavam misturados dentro do método de criação de pedido, foram transferidos para uma estrutura própria. Com isso, a lógica matemática de desconto deixa de competir por espaço com código SQL, validações de pagamento ou mensagens ao usuário. Essa separação torna a regra mais fácil de testar, alterar e reutilizar.

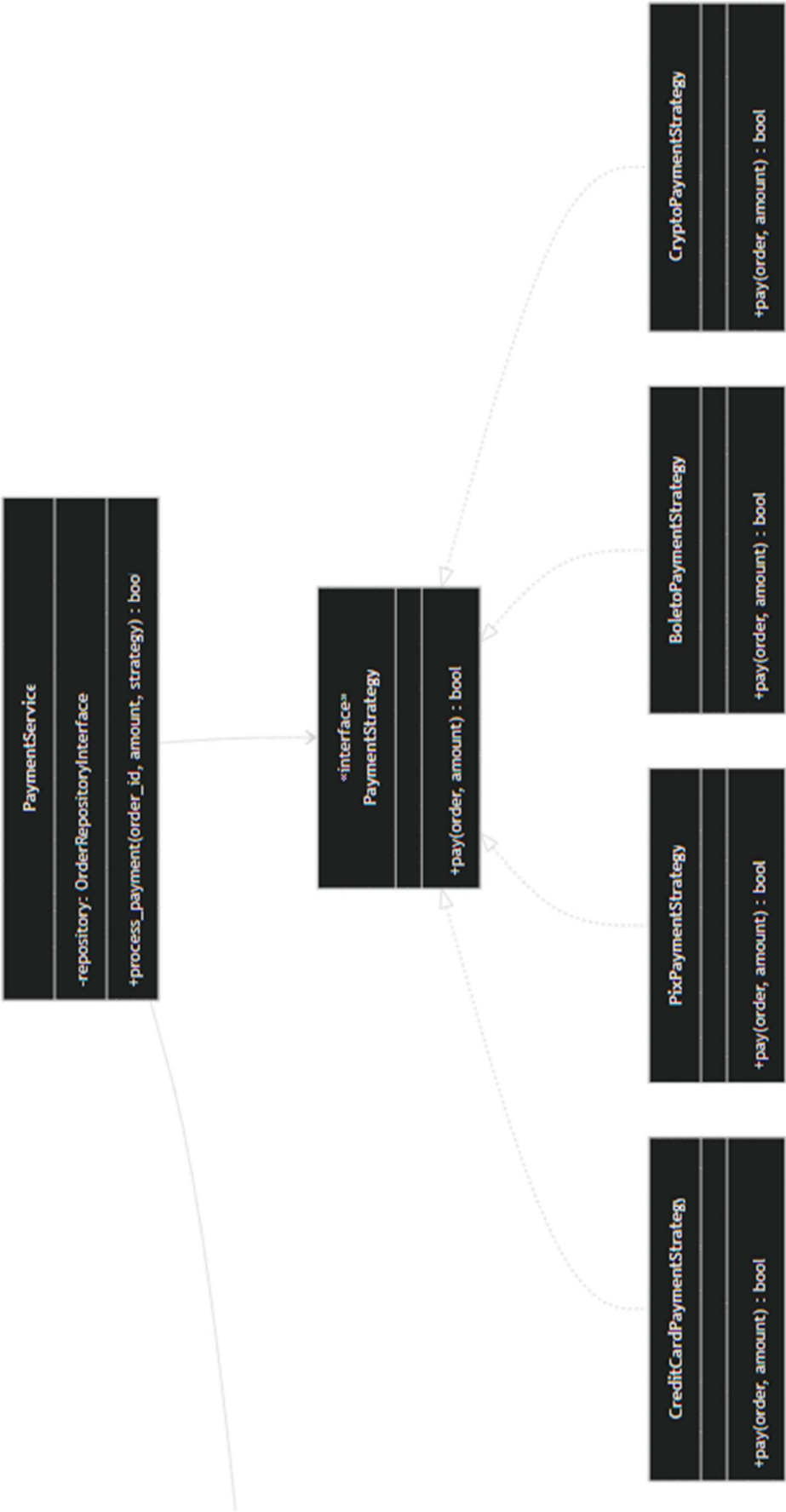
- **Separação da Comunicação com o Usuário:** as notificações, que antes eram simuladas diretamente com print() dentro da classe principal, passaram a ser representadas por componentes próprios. Assim, a responsabilidade de avisar o cliente sobre eventos do pedido fica isolada da responsabilidade de processar o pedido. Isso permite trocar futuramente uma notificação por console por e-mail, SMS ou WhatsApp sem modificar a regra central do sistema.

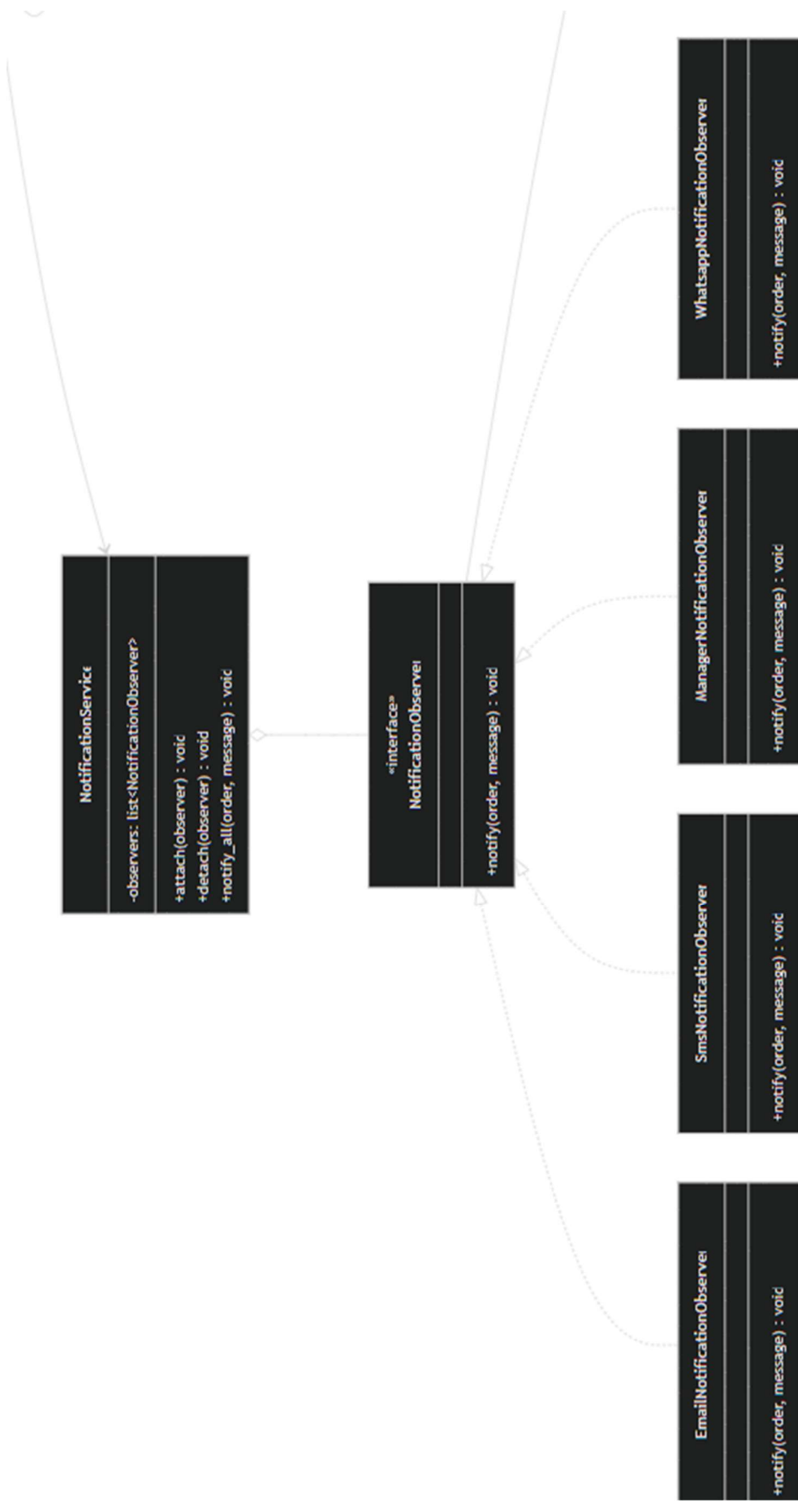
Com essa refatoração, o sistema deixou de depender de uma única classe gigantesca e passou a ter componentes pequenos, coesos e especializados. Cada classe agora possui um motivo claro para mudar: o repositório muda por causa da persistência, os modelos mudam por causa da estrutura dos dados, os serviços mudam por causa das regras de negócio e as notificações mudam por causa dos canais de comunicação.

Portanto, a violação de SRP foi corrigida por meio da decomposição da classe Sis em camadas e responsabilidades independentes, reduzindo o acoplamento, aumentando a legibilidade e permitindo que futuras alterações sejam feitas de forma localizada e mais segura.

3. Diagrama UML de Classes







4. Melhorias de Clean Code

Eliminação do Code Smell: Primitive Obsession (Obsessão por Primitivos)

O sistema utilizava múltiplos dicionários e strings soltas para representar pedidos, itens e estados. Exemplos:

```
{
  "nome": "produto1",
  "p": 100,
  "q": 2,
  "tipo": "desc10"
}
```

Dicionário representando um item no código legado

```
i['tipo'] == 'desc10'
```

Bastava errar uma única letra em uma chave string para quebrar o sistema em tempo de execução.

Como solução, criamos objetos de domínio ricos usando as dataclasses *Order* e *OrderItem* e mapeamos os comportamentos estritos usando os Enums *DiscountType*, *CustomerType* e *OrderStatus*.

```
@dataclass
class OrderItem:
    product_name: str
    unit_price: float
    quantity: int
    discount_type: DiscountType
```

```
class DiscountType(Enum):
    NORMAL = "normal"
    DISCOUNT_10 = "desc10"
    DISCOUNT_20 = "desc20"
    FREE_SHIPPING = "frete_gratis"
```

```
Orderitem.discount_type = DiscountType.DISCOUNT_10
```

Desta forma, o sistema passou a possuir maior segurança de tipos, reduzindo erros causados por strings mágicas e acessos inválidos a estruturas genéricas. Além disso, a modelagem explícita das

entidades aumentou a legibilidade, a manutenibilidade e a previsibilidade do código, permitindo que ferramentas de análise estática, como o mypy, validem inconsistências ainda em tempo de desenvolvimento.

Substituição por Nomes Significativos (Meaningful Names)

O sistema legado utilizava nomes excessivamente abreviados e pouco descritivos para variáveis, métodos e atributos. Exemplos:

```
def add_ped(self, n, its, t):  
    tot  
    st  
    dt  
    tp  
def proc_pag(self, id, m, vl):
```

Essas nomenclaturas dificultavam a compreensão do código, aumentavam a carga cognitiva durante manutenção e exigiam conhecimento prévio do funcionamento interno do sistema para interpretar corretamente cada elemento.

Além disso, nomes genéricos e abreviações excessivas reduziam a legibilidade e favoreciam interpretações ambíguas sobre o propósito das variáveis e funções.

Durante a refatoração, foi aplicado o princípio de *Meaningful Names*, substituindo identificadores genéricos por nomes explícitos e semanticamente descritivos. Dessa forma, o código passou a expressar melhor a intenção de cada elemento, tornando a leitura mais natural e reduzindo a necessidade de comentários explicativos.

```
unit_price  
quantity  
discount_type  
total_amount
```

Adicionalmente, foi adotado o uso de Type Hints para as funções. A assinatura da função se torna autoexplicativa: qualquer leitor do código consegue ler pelo cabeçalho da função os tipos de entrada como argumentos e o retorno esperado sem a necessidade de ler a implementação interna da função.

Como exemplo, em:

```
def calculate_total_amount_by_customer_name(self, customer_name: str) -> float:
```

Se torna trivial enxergar que a função espera uma string correspondente ao nome do cliente e que o retorno será um valor do tipo float, correspondente ao total de valores dos pedidos do cliente.

Além disso, Python é uma linguagem dinamicamente tipada. Ao se passar um objeto de tipo errado a uma função, o erro só ocorreria em tempo de execução (runtime), quebrando o sistema nas mãos do cliente. Ao adotar os Type Hints, o fluxo de trabalho de desenvolvimento passa por uma esteira de inspeção automatizada: passar objetos errados como parâmetros acaba gerando erros de ‘compilação’ antes mesmo do código ser executado.

5. Extensões Implementadas

Pagamento em Criptomoeda

A primeira extensão implementada consistiu na adição de um novo método de pagamento em criptomoeda, incluindo uma taxa adicional de 2% sobre o valor total do pedido. No sistema legado, a introdução de um novo meio de pagamento exigiria a modificação direta de estruturas condicionais extensas no método `proc_pag()`, aumentando o acoplamento e o risco de regressões.

Após a refatoração, o sistema de pagamentos passou a utilizar o padrão Strategy, permitindo que cada forma de pagamento fosse encapsulada em uma classe independente. Dessa forma, a nova funcionalidade foi implementada através da criação da estratégia `CryptoPaymentProcessor`, sem necessidade de alterar a lógica principal de processamento de pagamentos.

Diff conceitual da feature:

- Adicionado: `crypto_payment_processor.py`
- Adicionado: testes específicos para pagamento em criptomoeda
- Nenhuma alteração estrutural no fluxo principal de processamento

Trecho simplificado da implementação:

```
class CryptoPaymentProcessor(PaymentProcessor):  
    FEE_RATE = 0.02  
  
    def process(self, order: Order, paid_amount: float) -> bool:  
        total_with_fee = order.total_amount * (1 + self.FEE_RATE)  
  
        return paid_amount >= total_with_fee
```

A refatoração tornou essa extensão simples porque o comportamento de pagamento deixou de depender de condicionais centralizadas e passou a operar por polimorfismo através de estratégias intercambiáveis. O sistema principal permaneceu fechado para modificação, enquanto novas formas de pagamento puderam ser adicionadas por extensão.

Canal de Notificação via WhatsApp

A segunda extensão implementada foi a adição de um novo canal de comunicação via WhatsApp. No sistema legado, os envios de mensagens estavam diretamente acoplados à lógica principal da classe `Sis`, utilizando múltiplos blocos `if/elif` para controlar quais notificações seriam enviadas para cada tipo de cliente.

Com a refatoração, o sistema de notificações passou a utilizar o padrão `Observer`. O `OrderService` deixou de conhecer detalhes sobre envio de mensagens e passou apenas a disparar eventos de notificação. Cada canal de comunicação foi encapsulado em observers independentes.

Diff conceitual da feature:

- Adicionado: `whatsapp_notification_observer.py`
- Adicionado: `notification_factory.py`
- Adicionado: testes específicos para notificações WhatsApp
- Nenhuma alteração nas regras centrais de criação de pedidos

Trecho simplificado da implementação:

```
class WhatsAppNotificationObserver(NotificationObserver):  
  
    def notify(self, order: Order, message: str) -> None:  
        print(  
            f"WhatsApp enviado para "  
            f"{order.customer_name}: {message}"  
        )
```

A refatoração tornou essa extensão simples porque os canais de comunicação passaram a ser tratados como observers independentes e desacoplados da lógica de negócio. Dessa forma, novos canais puderam ser adicionados sem alterar os serviços principais do sistema.

Desconto Progressivo por Volume

A terceira extensão implementada foi o desconto progressivo por volume, responsável por aplicar 15% de desconto adicional em itens com quantidade igual ou superior a três unidades.

No sistema legado, a lógica de descontos estava rigidamente acoplada ao método `add_ped()`, através de estruturas condicionais responsáveis por interpretar manualmente os tipos de desconto dos itens.

Após a refatoração, as regras de desconto passaram a utilizar o padrão Strategy, permitindo encapsular cada política de desconto em classes independentes.

Diff conceitual da feature:

- Adicionado: `volume_discount_strategy.py`
- Adicionado: `discount_service.py`
- Adicionado: testes específicos para desconto por volume
- Nenhuma alteração no algoritmo principal de criação de pedidos

Trecho simplificado da implementação:

```
class VolumeDiscountStrategy(DiscountStrategy):  
  
    def apply(  
        self,  
        item: OrderItem,  
        current_total: float  
    ) -> float:  
  
        if item.quantity >= 3:  
            return current_total * 0.85  
  
        return current_total
```

A refatoração tornou essa extensão simples porque o cálculo de descontos passou a ser delegada a estratégias especializadas. Com isso, novas regras puderam ser adicionadas sem modificar o fluxo central do sistema, evidenciando o cumprimento do princípio Open/Closed.

6. Reflexão Metacognitiva

6.1. Princípios desafiadores ainda não dominados

O princípio SOLID que apresentou maior dificuldade de consolidação ao longo do desenvolvimento deste projeto foi o Open/Closed Principle (OCP). Embora a definição conceitual do princípio seja compreendida — entidades de software devem estar abertas para extensão, porém fechadas para modificação —, sua aplicação prática ainda levanta dúvidas relacionadas ao equilíbrio entre extensibilidade e simplicidade arquitetural.

Durante a refatoração do sistema legado, observou-se que a identificação de violações do OCP tornou-se relativamente evidente em trechos caracterizados pelo uso extensivo de estruturas condicionais (`if/elif`) para controlar regras de negócio, especialmente nos fluxos de pagamento, descontos e notificações. Entretanto, a principal dificuldade encontrada esteve na tomada de decisão sobre quais pontos do sistema realmente justificavam abstrações adicionais.

Em diversos momentos, surgiram questionamentos sobre até que ponto a introdução de factories, interfaces e estratégias especializadas agregaria flexibilidade real ao sistema ou apenas aumentaria sua complexidade estrutural. A experiência adquirida ao longo do projeto permitiu compreender que o objetivo do OCP não consiste em abstrair indiscriminadamente todos os componentes da aplicação, mas sim identificar os pontos com maior probabilidade de evolução futura e isolá-los adequadamente.

Nesse contexto, o projeto contribuiu para consolidar a percepção de que os mecanismos de pagamento, os canais de notificação e as regras de desconto representam pontos naturais de extensão, tornando justificável a utilização de padrões como Strategy, Observer e Factory Method.

6.2 Utilização de IA no desenvolvimento do projeto

A Inteligência Artificial foi utilizada como ferramenta de apoio técnico durante o processo de refatoração, principalmente para revisão arquitetural, identificação de violações dos princípios SOLID e discussão sobre possíveis aplicações de padrões GoF no contexto do sistema legado.

Os prompts que produziram resultados mais úteis foram aqueles formulados de maneira específica e contextualizada, direcionados a problemas concretos da aplicação, como a implementação de Observer para notificações, a aplicação de Strategy para regras de desconto e a organização da arquitetura em camadas. Esse tipo de interação permitiu utilizar a IA como

instrumento de apoio à tomada de decisão técnica, e não apenas como mecanismo de geração automática de código.

Por outro lado, prompts excessivamente genéricos, como solicitações amplas de “refatoração com SOLID” ou “melhoria completa do projeto”, produziram respostas menos úteis e frequentemente incompatíveis com as restrições reais do trabalho acadêmico, introduzindo abstrações excessivamente complexas para o porte do sistema.

Uma decisão de design em que houve discordância parcial em relação às sugestões fornecidas pela IA ocorreu durante a implementação do sistema de notificações. Algumas sugestões iniciais conduziam a uma estrutura arquitetural demasiadamente sofisticada para o escopo do projeto, com múltiplas camadas adicionais de abstração. Optou-se, portanto, por uma implementação mais simples da `NotificationFactory`, responsável apenas pela composição dinâmica dos observers conforme o tipo de cliente. Essa decisão preservou os princípios arquiteturais desejados — especialmente OCP, DIP e Observer — sem comprometer a legibilidade e a simplicidade do sistema.